

LC 1642: Furthest Building You Can Reach

Problem Analysis & Optimal Approach Strategy

Target Companies

DE-Shaw

Flipkart

Media.Net

OYO

Problem Breakdown

- **Condition 1:** If (current building size $\geq$ next building), no bricks or ladders are needed (it is a free move).
- **Condition 2:** If it is less, then we can use **one ladder**, or a **difference amount of bricks**.

Approach

Delay the ladder assignment. Save ladders for bigger climbs and spend bricks where the cost is least.

Strategy Steps:

- Initially give a ladder to all climbs.
- In case you exhaust the number of ladders, remove the ladder from the smallest climb and use bricks instead.
- A **Min-Heap (Priority Queue)** is ideal for tracking the smallest climbs we've used ladders on, enabling quick retrieval and replacement when ladder capacity is exceeded.

C++ Implementation

```
class Solution {
public:
    int furthestBuilding(vector<int>& heights, int bricks, int ladders) {
        int n=heights.size();
        int ans=0;

        priority_queue<int,vector<int> , greater<int>> minh;

        for(int i=0;i<n;i++){
            int diff=heights[i+1]-heights[i];
            if(diff<=0){
                continue;
            }

            minh.push(diff);
            if(minh.size()>ladders){
                bricks-=minh.top();
                minh.pop();
            }

            if(bricks<0){
                return i;
            }
        }
        return n-1;
    }
};
```